Economics of Scale: Floating Point vs Exact Integer ML Models

Why the Current Architecture Costs More, Delivers Less, and Cannot Improve, and What Replaces It

Registry: [HOWL-VDR-30-2026]

Series Path: [HOWL-VDR-1-2026] → [HOWL-VDR-2-2026] → [HOWL-MATH-3-2026] → [HOWL-MATH-4-2026] → … → [HOWL-VDR-14-2026] → … → [HOWL-VDR-21-2026] → [HOWL-VDR-22-2026] → [HOWL-VDR-23-2026] → [HOWL-VDR-24-2026] → [HOWL-VDR-25-2026] → [HOWL-VDR-26-2026] → [HOWL-VDR-27-2026] → [HOWL-VDR-28-2026] → [HOWL-VDR-29-2026] → [HOWL-VDR-30-2026]

DOI: 10.5281/zenodo.20270204

Date: May 2026

Domain: ML Infrastructure Economics / Exact Arithmetic

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

The ML industry is four years into commercial deployment. Datacenters costing hundreds of billions of dollars are under construction. The hardware going into those facilities, NVIDIA H100, B100, and their successors, contains integer execution units that deliver 2 × the throughput of the floating-point units the industry currently uses. The software stack running on that hardware discards information at every arithmetic operation, produces non-deterministic output, cannot secure data access structurally, and spends 80–95% of generated tokens on infrastructure that deterministic tools handle better.

This paper presents the economic analysis of VDR, Value, Denominator, Remainder, exact integer arithmetic as a replacement foundation for ML inference and training. VDR is not a competing model architecture. It is a different computational substrate: every number is an integer triple, every operation is exact, every result is deterministic, and the hardware to run it already exists in every modern GPU and CPU.

The economics compound across three independent axes. First, per-token throughput improves 1.5–2 × on existing hardware because INT8 tensor cores are faster than FP16 tensor cores. Second, total tokens per task reduce 70–98.6% because data lives in addressed knowledge bases, deduction runs in a Prolog engine, and grammars provide structural output, the LLM generates only judgment and prose. Third, cost scaling changes from

quadratic to linear because state is stored at integer addresses rather than re-read through the attention window every turn.

The compound effect for structured workloads: 40 × or greater reduction in total compute cost, with exact arithmetic, deterministic reproduction, and structural safety. For purely creative tasks, unconstrained prose, poetry, open conversation, VDR provides the per-token hardware speedup but minimal token reduction. The savings are task-shaped: the more structure a task has, the greater the reduction.

These numbers follow from instruction counts on published hardware specifications and measured token distributions across task categories. They do not require speculation about future hardware or unvalidated capabilities. The arithmetic is validated across 884 tests in 37 domains with zero computation errors. The hardware exists in production. The question is not whether exact integer ML is faster and cheaper. It is when the industry adopts it.

---

1. What This Paper Assumes You Know

Nothing about VDR. This paper is self-contained. It introduces each component as needed and derives economic consequences from mechanical properties. No prior paper in the series is required.

What this paper assumes about the reader: familiarity with the cost structure of running LLMs in production, per-token pricing, GPU utilization, memory bandwidth constraints, the engineering effort involved in reliability, safety, and deployment. The reader is assumed to understand what floating-point arithmetic is, what a transformer is, and what inference and training cost.

This paper does not advocate for immediate wholesale replacement of existing infrastructure. It presents a mechanical analysis of what exact integer arithmetic costs, what it delivers, and what it implies for infrastructure decisions being made now. The reader can evaluate the economics independently.

---

2. The Foundation: What VDR Is

A number in VDR is three integers: Value, Denominator, Remainder. Written [V, D, R]. The number it represents is (V + R) / D.

V is the integer numerator, what fits cleanly in the current denominator frame. D is the denominator, a fixed integer, shared across all values in a pipeline stage, never stored per element. R is the remainder, the exact portion of the value that V could not absorb within the D frame.

R is not rounding error. It is not noise. It is the part of the exact result that lives outside the current frame, stored as an integer rather than discarded. A VDR triple with R = 0 is

called closed, it represents the exact rational number V/D. A triple with R ≠ 0 is called active, it carries exact structure that the denominator frame couldn't absorb.

2.1 The divmod Rule

When any operation produces a result that doesn't fit in V at the current D, the system splits it using integer division and modulo.

Multiply two values sharing denominator D. Their numerators are p1 and p2. The exact product requires denominator D^2, which is larger than D. Instead:

```
product = p1  $\times$  p2

Q = product ÷ D         (integer division)

S = product mod D        (integer modulo)

result = [Q, D, S]
```

Q captures what fits. S captures exactly what doesn't. D stays fixed. Nothing is discarded.

When D is a power of two, and in this architecture it always is, this divmod reduces to a bit shift and a bitwise mask.

```
Q = product >> bits

S = product & mask
```

One shift. One mask. These are among the cheapest operations any processor executes. That is the entire cost of exact arithmetic beyond the multiply itself.

2.2 Why Power-of-Two Denominators

The denominator D is chosen as a power of two to match hardware register widths. For ML workloads, VDR assigns D = 2^8 (256) for weights, D = 2^16 (65536) for activations, and D = 2^64 for gradient accumulation during training. At these widths, divmod is native hardware, a shift and a mask, single-cycle operations on any processor manufactured in the last four decades.

The choice of D affects dynamic range and register packing. It does not affect exactness. The property that V + R always equals the true integer numerator holds at D = 2^8 exactly as it holds at any other power of two. The arithmetic is exact at every basis.

2.3 What Floating-Point Discards

A 32-bit float allocates 23 bits to its mantissa. Every operation whose true result requires more than 23 significant bits rounds. The IEEE 754 standard specifies how. It does not specify whether. It always rounds. This is the normal case.

For one operation the error is bounded at 0.5 ULP, half a unit in the last place. For N chained operations the errors accumulate. Under favorable conditions (random, uncorrelated rounding directions) the growth is approximately $\sqrt{N}$ ULP. Under unfavorable conditions (correlated operations, iterative algorithms, subtractive cancellation) the growth is linear in N or worse.

VDR has no per-operation error. The sum V + R always equals the exact numerator. Chain length is irrelevant to accumulated error because there is no per-step error to accumulate.

2.4 Validation

The VDR arithmetic system has been validated across 884 tests in 37 domains, 23 mathematical (number theory, polynomial algebra, linear algebra, signal processing, differential equations, optimization, probability, cryptographic primitives, chaos theory, graph theory, game theory, and others) and 14 physical (QED, quantum mechanics, orbital mechanics, optics, thermodynamics, and others). Zero VDR computation errors. All 14 test failures in the full suite were traced to test-design errors, wrong expected values, overly tight thresholds, not to incorrect arithmetic.

This is not a claim that the system handles every possible computation. It is a statement that across every computation tested, the arithmetic produced exact results. The errors, where they occurred, were in the tests, not in the math.

---

3. Hardware: Why VDR Is Faster on Existing Silicon

The performance advantage of VDR over float is not theoretical. It follows from published hardware specifications of GPUs and CPUs currently in production and currently being installed in the datacenters under construction.

3.1 The H100 Execution Units

The NVIDIA H100 GPU, the dominant chip in current datacenter builds, contains several types of execution unit, each with different throughput. The numbers that matter, per streaming multiprocessor per clock cycle:

Execution unit	Throughput (ops/SM/cycle)	Used for
FP16 tensor cores	512	current ML matrix multiply
INT8 tensor cores	1024	VDR matrix multiply
FP32 CUDA cores	128	current general float ops
INT32 CUDA cores	64	VDR general integer ops
Special Function Unit (SFU)	32	exp, log, rsqrt, sin, cos, division

Two numbers dominate. INT8 tensor cores run at 2 × the throughput of FP16 tensor cores, 1024 versus 512 operations per SM per cycle. The SFU, which processes every transcendental function in a float pipeline (exponentiation for softmax, tanh for GeLU, reciprocal square root for layer normalization), runs at 1/16 the FP16 tensor core rate.

VDR uses INT8 tensor cores for matrix multiply and replaces every SFU operation with a table lookup in shared memory or an integer arithmetic sequence. The SFU bottleneck, which constrains every float pipeline, does not exist in VDR.

3.2 Matrix Multiply (GEMM)

Matrix multiplication dominates transformer compute. It accounts for 85–95% of the floating-point operations in a forward pass.

FP16 tensor cores process 512 fused multiply-adds per SM per cycle, producing FP32 accumulator outputs. Each output is rounded to fit the FP32 mantissa.

INT8 tensor cores process 1024 integer multiply-adds per SM per cycle, producing INT32 accumulator outputs. Each output is exact, INT32 holds the true integer sum. After the tile computation completes, an epilogue kernel applies a right shift and mask to split each result into V and R. This epilogue costs two instructions per output element.

Metric	FP16 tensor core	VDR INT8 tensor core
Operations per SM per cycle	512	1024
Peak throughput (132 SMs, H100)	~990 TFLOPS	~1980 TOPS
Accumulator	FP32 (23-bit mantissa, rounds)	INT32 (32-bit, exact)
Projected utilization (current kernels vs new)	85–95%	75–85%
Effective throughput	~840–940 TFLOPS	~1485–1685 TOPS
Projected speedup		**1.6–1.8 ×**

The 2 × raw hardware advantage is partially offset by kernel maturity. cuBLAS has had years of optimization; VDR kernels are new. The 75–85% utilization estimate is conservative for first-generation implementations. As kernels mature, effective throughput approaches the full 2 × hardware ratio.

3.3 Softmax

Softmax converts logits to probabilities. It requires exponentiation, summation, and division, all SFU-bottlenecked in float.

In float: exponentiation via the SFU at 32 ops/SM/cycle. Division via the SFU at 32 ops/SM/cycle. These are 1/16 the tensor core rate and dominate softmax execution time.

In VDR: exponentiation via table lookup in shared memory. Because inputs are bounded integers, the table for the relevant range fits in 4–16 KB of shared memory. One memory load per element at full bandwidth. Division via Barrett reduction, a precomputed multiplicative inverse applied as an integer multiply and shift. Full INT32 core throughput.

Metric	Float	VDR
Exp throughput	SFU: 32 ops/SM/cycle	shared memory load: full bandwidth
Division throughput	SFU: 32 ops/SM/cycle	Barrett: INT32 core rate
Result precision	polynomial approximation	exact
Output sum	≈ 1	$= 1$
Projected speedup		**3–4** $\times$

3.4 Activation Functions and Layer Normalization

GeLU, SiLU, and other activation functions require tanh or sigmoid, SFU operations. Layer normalization requires reciprocal square root, an SFU operation.

In VDR: all replaced by table lookups. For Q8 inputs, the table is 256 entries $\times$ 4 bytes = 1 KB, trivially fitting in shared memory. For Q16 inputs with bounded activation range, 4–16 KB.

Operation	Float	VDR	Speedup
GeLU / SiLU	SFU polynomial, 32 ops/SM/cycle	table lookup, full bandwidth	4–6 $\times$
Layer norm rsqrt	SFU, 32 ops/SM/cycle	table or integer Newton	2–3 $\times$
Layer norm division	float division	bit shift (power-of-two hidden dims)	3–4 $\times$

3.5 Memory Bandwidth

H100 HBM3 bandwidth: 3.35 TB/s. Single-batch autoregressive inference, one token at a time, is memory-bandwidth-bound. The GPU waits for weight data to arrive from memory, not for computation to finish.

FP16 weights: 2 bytes per parameter. A 7B parameter model occupies 14 GB. VDR INT8 weights: 1 byte per parameter (V only, R is zero for frozen weights and is not stored). The same model occupies 7 GB.

Metric	FP16	VDR INT8
Bytes per weight	2	1
Model size (7B)	14 GB	7 GB
Minimum load time per token	4.2 ms	2.1 ms
Effective bandwidth advantage		**2** $\times$

When inference is memory-bound, half-size weights translate directly to approximately 2

× throughput.

3.6 Full Forward Pass

Combining all operations for a 7B parameter model, 32 transformer layers, hidden dimension 4096, 32 attention heads, sequence length 2048, single-batch inference on one H100:

Component	FP16 (ms/layer)	VDR INT8 (ms/layer)	VDR speedup
QKV projection	0.83	0.42	2.0 ×
Attention GEMM	0.55	0.28	2.0 ×
Softmax	0.036	0.009	4.0 ×
Attention output projection	0.28	0.14	2.0 ×
FFN up + gate	1.10	0.55	2.0 ×
GeLU activation	0.008	0.002	4.0 ×
FFN down projection	0.55	0.28	2.0 ×
Layer norm (× 2)	0.008	0.004	2.0 ×
Residual add (× 2)	0.002	0.003	0.7 ×
Full forward pass	**~110 ms**	**~55 ms**	**~2 ×**

Residual addition is the single operation where float is faster, integer addition with carry propagation costs more than a single float add. This operation contributes less than 0.1% of total forward pass compute.

3.7 Where These Numbers Come From

Every number in this section derives from published hardware specifications and instruction counts. The tensor core throughputs are NVIDIA's published numbers. The SFU throughput is NVIDIA's published number. The memory bandwidth is NVIDIA's published number. The cycle counts for shift, mask, add, and multiply are published in vendor documentation.

The projected utilization ranges (75–85% for VDR versus 85–95% for float) reflect kernel maturity differences. The VDR projections are conservative, they assume first-generation kernels that have not undergone the years of optimization that cuBLAS represents. The hardware headroom is known and the engineering path to higher utilization is standard.

---

4. Tokens: Why VDR Uses Fewer

The per-token speedup from Section 3 is one axis of cost reduction. The second axis, and the larger one, is that VDR eliminates most tokens entirely.

4.1 What Current LLMs Spend Tokens On

Examine any production LLM conversation performing a structured task. The tokens fall into categories:

Arithmetic performed as text. The model writes out calculations, carries digits, sometimes gets them wrong. Every arithmetic token is work that an integer ALU does in nanoseconds with exact results.

State reconstruction. Each turn, the model re-reads the conversation history through the attention window to remember what was established earlier. The longer the conversation, the more tokens spent re-establishing context that hasn't changed.

Data serialization. Information encoded as JSON, markdown tables, formatted text. The model generates structural tokens, braces, brackets, commas, field names, that carry no information beyond format.

Deductive reasoning performed as prose. "If A and B, then C. Since we established A in the previous step, and B follows from…", logical derivation written as English sentences, token by token, when a logic engine performs the same deduction deterministically in microseconds.

Hedging and safety language. "It's important to note that…", "I should mention…", "Please consult a professional…", tokens that exist for liability and alignment, not information.

Formatting and presentation. Headers, bullet points, numbered lists, code blocks, emphasis markers. Structural tokens that a grammar generates for free.

Measured across task categories, these infrastructure tokens account for 70–98.6% of total output, depending on the task.

4.2 The VDR-LLM-Prolog Architecture

VDR is the arithmetic layer. Above it sits a complete system architecture that eliminates infrastructure tokens structurally.

Knowledge bases (KBs) store data as facts at integer addresses. Data is not serialized into the token stream. The LLM references it by address. A dotted path like `root.sre.incident_42.timeline` resolves to two integers, a KB ID and a slot ID, at O(1) access cost. No tokens consumed.

A Prolog engine performs deduction. Logical rules are stored in KBs. The engine evaluates them via depth-first search with backtracking over exact integer values. Unification uses exact comparison, cross-multiplication of exact integers, not epsilon comparison of floats. Deduction that would cost hundreds of tokens as prose costs zero tokens as Prolog evaluation.

448 typed builtins across 25 categories handle computation. 173 numeric builtins cover arithmetic, linear algebra, number theory, probability, statistics, polynomial operations, finite fields, graph math, and more. All execute as exact integer operations. The LLM

selects which builtin to invoke, approximately 8 tokens per invocation, rather than generating the computation as text.

Grammars provide structural tokens. A grammar declares the fixed structure of an output format and leaves typed slots for content. The structural tokens (JSON braces, table headers, markdown formatting) are generated by the grammar at zero LLM cost with 100% correctness. The LLM fills only the content slots.

Scoped sessions with snapshot and clone manage state. A session's state is captured atomically, cloned for parallel exploration, and killed when no longer needed. State persists in KBs across turns. The LLM does not re-read history, it reads current state from KB addresses.

4.3 What the LLM Actually Does

In this architecture the LLM does three things:

Orchestration. It assesses the current state (by reading KB facts), decides what to do next, selects which builtins or Prolog queries to invoke, and interprets results. This is judgment, deciding what question to ask, what tool to use, what to investigate next.

Prose generation. When the output requires natural language, explanations, summaries, recommendations, creative writing, the LLM generates it. If a grammar defines the output structure, the LLM fills content slots. If the output is unconstrained prose, the LLM generates freely.

Novel rule creation. When the LLM encounters a pattern it recognizes, it can formalize it as a Prolog rule and store it in a KB. The rule executes at zero LLM token cost on all future encounters. The system gets smarter with use.

Everything else, arithmetic, data retrieval, deduction, formatting, state management, access control, is handled by deterministic components that consume zero LLM tokens.

4.4 Token Reduction by Task Category

The savings depend entirely on how much of a task is infrastructure versus judgment. This is not a single number. It is task-shaped.

Task category	Token reduction	Why
SRE investigation	98.6%	almost entirely log parsing, metric comparison, timeline reconstruction, threshold checking, causal deduction
Legal document review	96.2%	clause matching, regulatory comparison, obligation tracking, deadline extraction

Task category	Token reduction	Why
Financial analysis	96%	arithmetic, time series, threshold checking, compliance verification
Medical record analysis	94.1%	lab value comparison, medication interaction checking, guideline matching
Codebase migration	93.3%	AST analysis, pattern matching, dependency tracking, test generation
Customer support	70%	more prose, but still substantial state management and knowledge retrieval
Grading and evaluation	71.4%	rubric application is structured; feedback prose is not
Open conversation	10–30%	grammar-assisted formatting only; content is pure LLM
Poetry	~0%	every token is creative judgment; VDR adds nothing

The 85–97% range cited in summary materials refers to the structured task categories that dominate commercial LLM usage. For purely creative tasks, token reduction is minimal. The per-token hardware speedup (Section 3) still applies in all cases.

4.5 How Token Reduction Works Mechanically

Consider an SRE investigation that currently costs 25,100 tokens in a conventional LLM.

The conventional model reads the incident description, generates a plan as prose, requests log data (formatted as JSON in the token stream), parses the logs (as text, token by token), performs threshold comparisons (as arithmetic in prose), correlates timestamps (as text manipulation), deduces causation (as a chain-of-thought paragraph), formats findings (as markdown), and adds hedging and caveats.

In VDR: the incident description is stored as KB facts. The investigation plan is a Prolog rule that fires builtins in sequence. Log data is loaded into KB data primitives, LRU caches, queues, ring buffers, at integer addresses. Threshold comparisons are exact integer comparisons via builtins. Timestamp correlation is an integer sort. Causal deduction is a Prolog query over stored rules. Findings are output through a grammar that provides table structure. The LLM generates one paragraph of natural language assessment.

Total: 769 tokens. The LLM generated the assessment paragraph and the orchestration decisions. Everything else was deterministic, exact, and free of LLM token cost.

25,100 tokens versus 769. The factor is not 2 × or 5 × . It is 33 × . And the VDR result is exact where the conventional result contains approximate arithmetic, potential hallucination, and non-deterministic output.

4.6 Scaling: Quadratic vs Linear

The token reduction per turn is one effect. The scaling behavior over multiple turns is a separate and larger effect.

In a conventional LLM, every turn re-reads the entire conversation history through the attention window. Turn 1 processes T tokens. Turn 2 processes 2T. Turn 10 processes 10T. Turn 100 processes 100T. The total token cost for a 100-turn conversation is proportional to T × (1 + 2 + 3 + … + 100) = T × 5050. This is quadratic growth.

In VDR, state is stored in KBs. Each turn reads current state from KB addresses, a fixed-cost operation independent of conversation length. Turn 1 costs C tokens. Turn 100 costs C tokens. The total for 100 turns is 100C. This is linear growth.

Conversation length	Conventional (relative cost)	VDR (relative cost)	Ratio
Turn 1	1	1	1:1
Turn 10	55	10	5.5:1
Turn 20	210	20	10.5:1
Turn 50	1,275	50	25.5:1
Turn 100	5,050	100	50.5:1

This table shows relative scaling only, it does not yet include the per-turn token reduction from Section 4.4. When both effects compound (fewer tokens per turn and linear rather than quadratic scaling), the cost advantage for long structured conversations reaches ratios of 100:1 to 600:1.

---

5. The Compound Economics

The per-token speedup (Section 3) and the token reduction (Section 4) are independent. They multiply.

5.1 Cost Composition

Take a structured task, an SRE investigation, legal review, or financial analysis, that currently costs $27.58 at standard API pricing.

Token reduction: 25,100 tokens become 769 tokens. Cost at the same per-token rate: $0.85.

Per-token speedup: each token is generated ~2 × faster. The hardware serves 2 × the tokens per second. The effective per-token cost halves. $0.85 becomes $0.42.

Scaling over session: if the investigation spans 20 turns, conventional cost grows quadratically while VDR cost grows linearly. The per-turn savings compound.

The measured end-to-end result for SRE: $0.39 versus $27.58. A factor of 71 × .

This is not a projection. It is an instruction count (hardware speedup) multiplied by a measured token distribution (token reduction) applied to a real task structure. Each component is independently verifiable.

5.2 Cost by Task Category

Task	Conventional cost (relative)	VDR cost (relative)	Factor
SRE investigation	100%	1.4%	71 ×
Legal document review	100%	2.6%	38 ×
Financial analysis	100%	2.8%	36 ×
Medical record analysis	100%	4.1%	24 ×
Codebase migration	100%	4.7%	21 ×
Customer support	100%	15%	6.7 ×
Open conversation	100%	45–55%	1.8–2.2 ×
Creative writing	100%	50–60%	1.7–2 ×

The floor is the per-token hardware speedup, approximately 2 × , which applies even when zero tokens are eliminated. The ceiling depends on task structure. Most commercial LLM workloads fall in the 20–70 × range.

5.3 What This Means at Datacenter Scale

A datacenter serving 1 million requests per day on structured workloads at a blended 30 × cost reduction:

Metric	Conventional	VDR
GPU-hours per day	10,000	333
GPUs required (at utilization)	~420	~14
Annual GPU cost ($30K/GPU/year)	$12.6M	$420K
Annual energy (at 700W/GPU)	2,570 MWh	86 MWh
Annual energy cost ($0.10/kWh)	$257K	$8.6K

The same workload. The same quality, actually higher quality, since arithmetic is exact and output is deterministic. 30 × fewer GPUs. 30 × less energy. 30 × lower cost.

These are not speculative figures. They are the hardware speedup multiplied by the token reduction applied to representative task distributions. A facility operator can perform this calculation against their own workload mix today.

5.4 The Savings Are Conservative

These calculations assume first-generation VDR kernels at 75–85% hardware utilization. Mature kernels at 90–95% utilization would improve the per-token speedup from ~2 × toward the full hardware ratio. They assume no improvement in token reduction from system maturation, but VDR systems improve with use as Prolog rules accumulate, further reducing LLM token requirements over time. The 30 × blended estimate is a lower bound, not a central estimate.

---

6. What Floating-Point Cannot Fix

The economics in Section 5 are the quantitative argument. This section presents the qualitative argument: the problems that floating-point arithmetic creates in ML systems are properties of the representation, not engineering failures that scale or optimization can address.

6.1 Non-Determinism

Floating-point addition is not associative. (a + b) + c does not always equal a + (b + c). On a GPU, threads within a warp accumulate partial sums. Thread scheduling is non-deterministic. The order of additions varies between runs. The result changes.

This means: the same model, with the same weights, processing the same input, produces different logits on consecutive runs. Usually the difference is small enough that the argmax selects the same token. At decision boundaries, where two tokens have similar probability, the selection flips between runs.

This is not a bug to be fixed. It is a mathematical property of the representation. Floating-point addition will never be associative. No amount of engineering makes it so. Integer addition is associative. VDR produces bit-identical results across runs, across platforms, across hardware generations.

The engineering consequences: you cannot write deterministic tests for LLM output. You cannot build reliable automated pipelines that depend on consistent output. You cannot perform clean A/B testing because arithmetic non-determinism contaminates the signal. You cannot provide regulatory audit trails that demonstrate reproducible computation. Every evaluation benchmark carries noise from arithmetic, not just from the model.

VDR eliminates all of these problems structurally. Same input, same output, every time, on any hardware.

6.2 Softmax Does Not Sum to 1

Every token selection in an LLM passes through softmax. In float, the output distribution sums to approximately 1. The approximation error is small per operation but systematic. The probability mass assigned to each token is slightly wrong. Sampling decisions at probability boundaries, where two tokens have nearly equal likelihood, can select the wrong token.

In autoregressive generation, each token conditions on all previous tokens. A single wrong selection at position 50 produces a completely different sequence by position 500. Not slightly different. Different.

In VDR, softmax sums to exactly 1. Not approximately. The probability distribution is exact. Sampling boundaries are at their true positions. The exact rational surrogate softmax, (shifted input)^2 / sum of (shifted inputs)^2, is non-negative, monotonic, differentiable, and sums to exactly 1/1. It has been validated through end-to-end training producing measurably lower loss across epochs.

6.3 Attention Degrades with Context Length

Attention computes weighted averages where the weights come from softmax over query-key dot products. The dot product accumulates across the hidden dimension, typically 4096 multiply-adds, accumulated in whatever order the hardware schedules. The result is exponentiated (amplifying small differences), normalized by a sum (redistributing error), and used to weight another accumulation.

As context windows grow, 128K, 1M, 10M tokens, softmax sums over longer sequences. Individual attention weights become smaller. The numerical distinction between "attend to this token" and "ignore this token" becomes a smaller fraction of the representable range. Float's resolution is fixed while the dynamic range of meaningful differences shrinks.

This suggests that some portion of the "lost in the middle" phenomenon, the documented failure of LLMs to use information in the middle of long contexts, may be arithmetic, not architectural. The model may have learned to attend to the right token, but float noise in the attention computation may obscure the signal. Nobody can currently distinguish "the model didn't learn to attend" from "the model learned to attend but the arithmetic lost it," because there is no exact-arithmetic baseline to compare against.

VDR would provide that baseline. Whatever attention failures remain under exact arithmetic are definitively model limitations, not arithmetic artifacts.

6.4 Long Chains Drift

Every operation in a float pipeline rounds. In a chain of operations, each step operates on the rounded output of the previous step. Errors compound.

For LLM inference, a 70B model with 80 transformer layers processes each token through 80 attention computations and 80 activation functions, chained sequentially. For a 10,000-

token reasoning chain, that is roughly 1.6 million sequential floating-point-heavy operations per latent element.

For diffusion models generating video, a 2-hour film at 24 fps with 150 denoising steps per frame chains 25.9 million arithmetic operations per latent element. At this chain length, float64 drift reaches approximately 2.6×10^{-7} per element. This manifests as color shift and temporal flicker, artifacts that are arithmetic in origin, not model error.

Float drift in diffusion cannot be solved by scaling or engineering. The rounding is intrinsic to the representation. The only mitigations are periodic correction passes (which introduce discontinuities and cost 5–8% overhead) and acceptance of the drift (which limits output quality for long-form content).

VDR drift is zero at any chain length, by construction. Integer shift and mask are exact operations. There is no mechanism by which error can enter. A 2-hour video generated with VDR arithmetic has the same per-element fidelity at the last frame as the first frame.

6.5 Special Values

Floating-point arithmetic can produce infinity (from overflow), NaN (from 0/0 or ∞ - ∞), and denormals (values near zero that trigger slow-path hardware handling). NaN propagates silently through all subsequent computation, corrupting results without signaling an error. Denormals cause unpredictable performance degradation or are flushed to zero (silent information loss).

Production float pipelines include defensive code throughout: isnan checks, gradient clipping, loss scaling, epsilon additions to denominators, flush-to-zero mode configuration. This code executes on every step whether needed or not. It exists because the representation can produce values that break downstream computation.

Integer arithmetic has no NaN, no infinity, no denormals, no negative zero. Overflow is preventable by basis selection, verified at model load time, not checked at runtime. The entire category of runtime numerical defense does not exist in VDR. No epsilon parameters, no loss scaling, no gradient clipping for numerical stability, no flush-to-zero configuration.

6.6 The Wall

These problems do not improve with scale. A larger model does not make float addition associative. More training data does not make softmax sum to 1. A longer context window does not reduce the accumulated drift per element. More parameters do not eliminate NaN propagation.

The industry is investing in scale, more parameters, more data, more compute, and experiencing diminishing returns. Each increment costs more and delivers less. This is often framed as a research problem: the scaling laws are bending, new architectures are needed, new training methods are required.

Some of the diminishing returns are genuine research problems. But some are the float wall. A model that gets better at arithmetic through scale is still worse at arithmetic than

integer division. A model that hallucinates less through better training still hallucinates more than a system that retrieves facts by address. A model that maintains coherence better over long contexts through architectural innovation still degrades faster than a system that stores state at integer addresses.

Improving the model is valuable. But the model is running on a substrate that silently corrupts its output, non-deterministically, at every step, and no improvement to the model changes that. The substrate is the constraint.

---

7. What VDR Opens: New Capabilities

The economic argument, cheaper, faster, more exact, is sufficient motivation for adoption on existing workloads. But VDR also enables workload categories that float cannot serve.

7.1 Deterministic LLM Applications

Any application requiring reproducible output: regulatory compliance (financial, medical, legal), automated testing of LLM-integrated systems, cached inference where recomputation must match the original, audit trails demonstrating that a specific input produced a specific output.

Currently impossible because float non-determinism means the same input can produce different output on consecutive runs. Under VDR: guaranteed. Same input, same output, every time, on any hardware. This is not a quality level to be achieved. It is a structural property of integer arithmetic.

7.2 Long-Form Generative Media

Feature-length video, continuous livestream generation, multi-day simulation. Any workload chaining millions of arithmetic operations per element.

Currently limited by float drift: correction passes introduce discontinuities, drift-induced artifacts accumulate, temporal coherence degrades. Under VDR: unlimited chain length with zero drift, zero correction passes, bit-identical reproducibility. The same scene rendered on different machines composites with zero numerical seam.

The commercial trajectory of generative media points toward longer output. Every step in that direction amplifies VDR's advantage and hardens float's wall.

7.3 LLM Software

The VDR architecture introduces a category: software developed through conversation, deployed as snapshots, improved by usage.

A session configured with knowledge base state and Prolog rules is snapshot and cloned as a running application. The clone is an instance, same persistent knowledge, independent working state. Clones serve requests, accumulate experience as rules, and are killed when

stale. Development cost for a simple chatbot: 4 hours versus 2–4 weeks conventional. For an SRE assistant: 12 hours versus 6–12 weeks.

Three execution levels that applications mature through: L1 (full LLM judgment, 50–500 tokens per operation), L2 (LLM invokes stored Prolog rules, 8 tokens), L3 (pure Prolog batch, zero tokens). Investigation 1 at a given task might be 100% L1, full LLM judgment on every step. Investigation 100 might be 75% L3, three quarters of the work handled by rules the system learned from prior investigations, no LLM involvement. By investigation 100, cost per investigation has dropped 83%.

This is a new application development paradigm. It requires exact state (for snapshot fidelity), deterministic logic (for rule execution), addressed data (for KB persistence), and structural safety (for deployment without behavioral guardrails). VDR provides all four as consequences of its arithmetic foundation.

7.4 LLM Server Software

Extending the application pattern to network services. Protocol grammars speak wire formats, HTTP, SMTP, DNS, MQTT, SSH, and 30+ others. Grammars provide all structural tokens at zero LLM cost. Prolog rules process requests. Clones spawn per connection for isolation. The LLM fills content slots and provides judgment when no stored rule matches.

Security is structural: authentication is credential facts plus Prolog evaluation, authorization is the grant system, rate limiting is exact integer counter comparison (no float drift causing false threshold crossings), SQL injection has no vector (Prolog queries are typed), XSS is impossible (grammars produce safe output by construction).

Financial accuracy is structural: $10,000.00 is [1000000, 100, 0]. 99.99% SLA is [9999, 10000, 0]. No rounding. No float accumulation drift. No approximate equality.

7.5 Structural Safety

VDR's safety model deserves explicit economic accounting because it replaces an entire cost center.

Current safety infrastructure: RLHF training, red team exercises, prompt injection defense, output filtering, content moderation, jailbreak patching, alignment tax on every response. These are ongoing operational costs that scale with usage.

VDR's safety is structural: the LLM never receives unauthorized data because KB visibility (integer comparison) and scope chain (ancestor walk) filter before the LLM is involved. No prompt modifies any integer in any access control check. Session identity is set at authentication. Three independent layers, input filtering, grant authorization, output validation, must all fail simultaneously for a breach.

What prompt injection can do: influence which builtins the LLM invokes and how it phrases prose. What it cannot do: change the user ID, modify the scope chain, bypass visibility checks, execute operations without grants, or surface data the session is not authorized to see.

The cost of this safety is zero LLM tokens. No hedging language, no safety disclaimers, no refusal reasoning, no alignment tax on response quality. Safety is a property of the architecture, not a behavior trained into the model. The annual cost of safety-related engineering, training, red teaming, and operational monitoring is replaced by the one-time cost of implementing integer comparison and scope chain traversal.

---

8. Adoption Economics

8.1 What Switching Costs

VDR runs on existing hardware. No new GPUs, no new datacenters, no new silicon. The INT8 tensor cores, INT32 CUDA cores, and shared memory that VDR uses are already present in every H100 and every GPU since NVIDIA's Volta generation (2017).

The switching cost is software:

Kernel library. The GEMM, softmax, activation, layer normalization, and residual kernels described in this paper. Conservative estimate for functional implementation: 3–4 months of engineering. For competitive-with-cuBLAS optimization: 12–18 months.

Operator framework. Integration with or replacement of the PyTorch/JAX operator dispatch. This is the layer that routes operations to the correct kernel.

Model conversion. Converting existing trained models from float weights to VDR INT8 weights. This is a one-time operation per model, mechanically similar to existing INT8 quantization, with the addition that VDR stores the remainder rather than discarding it.

KB and Prolog infrastructure. For the full token-reduction architecture. This is new software, knowledge base storage, Prolog engine, grammar system, session management. The implementation blueprint specifies 65 modules, approximately 20,500 lines, in 5 incremental stages.

Training infrastructure. For training new models natively in VDR arithmetic. This requires exact-fraction forward pass, backward pass, and optimizer, all validated in the Python prototype at small scale.

8.2 Incremental Adoption Path

VDR does not require wholesale replacement. It can be adopted incrementally, with each stage delivering independent value.

Stage 1: INT8 inference with remainder. Take existing quantized INT8 inference pipelines. Instead of discarding the quantization remainder, store it. Run the same tensor core operations. Apply shift and mask epilogue. Cost: kernel modifications. Benefit: exact arithmetic on existing INT8 infrastructure, ~2 $\times$ throughput from tensor core advantage, zero drift.

Stage 2: Table lookup for transcendentals. Replace SFU-bottlenecked softmax, GeLU, and layer norm with shared memory table lookups. Cost: new kernels for these

operations. Benefit: 3–6 × speedup on SFU-limited operations, exact results.

Stage 3: Knowledge base integration. Add addressed data storage alongside the LLM. Data loaded into KBs rather than serialized through the token stream. Cost: KB infrastructure. Benefit: token reduction begins, proportional to the fraction of tokens currently spent on data serialization.

Stage 4: Prolog and grammar integration. Add logic engine and grammar-directed output. Cost: Prolog engine, grammar system. Benefit: full token reduction for structured tasks, structural safety.

Stage 5: Native VDR training. Train models from scratch in exact integer arithmetic. Cost: training infrastructure. Benefit: the first models whose training is fully exact and fully reproducible.

Each stage is independently valuable. Each runs on existing hardware. No stage requires commitment to subsequent stages. A facility operator can adopt Stage 1 tomorrow with kernel modifications and capture the ~2 × throughput advantage on existing INT8 workloads.

8.3 Risk Asymmetry

The risk of investigating VDR: a small engineering team for a few months. The kernels run on existing hardware. If the economics don't materialize in practice, the investment is a few engineer-months.

The risk of not investigating: a competitor deploys VDR and serves the same workloads at 2–5% of the cost. Your capital investment in GPU infrastructure is producing 30–70 × less value per dollar than theirs. You cannot respond by optimizing float, the advantage comes from architectural elimination of work, not faster execution of the same work.

The downside of checking is bounded. The downside of not checking is unbounded. This asymmetry should be sufficient to motivate investigation regardless of prior beliefs about VDR's viability.

---

9. What VDR Does Not Change

9.1 Model Quality Is Model Quality

VDR makes arithmetic exact. It does not make models smarter. A model that produces poor judgments produces poor judgments exactly. A model with insufficient training produces insufficient-quality output deterministically.

The improvement is that arithmetic errors no longer contaminate the assessment of model quality. When output is wrong under VDR, it is wrong because the model is wrong, not because the arithmetic drifted or the attention computation lost precision or the softmax probabilities were slightly misallocated. This makes model improvement more tractable,

you can identify and address the actual source of errors without filtering out arithmetic noise, but it does not substitute for model improvement.

9.2 Creative Tasks See Minimal Token Reduction

Poetry, fiction, open-ended conversation, brainstorming, emotional support, tasks where the LLM's entire contribution is language, see little or no token reduction. Every token is judgment and prose. There is no infrastructure to eliminate.

These tasks still benefit from the per-token hardware speedup (~2 $\times$) and from exact arithmetic in the underlying attention and softmax computations. But the 20–70 $\times$ cost reductions are specific to structured tasks. The economics are task-shaped. Any projection of cost savings must account for the actual task distribution of the workload.

9.3 Kernel Maturity Is Real

The per-token speedup projections assume first-generation VDR kernels at 75–85% hardware utilization. cuBLAS achieves 85–95% after years of optimization. The gap is real. The projections in this paper are net of this gap, they represent what first-generation implementations deliver, not what the hardware could theoretically provide.

Closing the gap is standard kernel engineering: tiling optimization, warp scheduling, register allocation, autotuning. The engineering path is known. The timeline is 12–18 months for competitive utilization levels. The projections in this paper do not assume this optimization has occurred.

9.4 Ecosystem Cost Is Real

The current ML ecosystem, PyTorch, JAX, TensorFlow, ONNX Runtime, TensorRT, hundreds of supporting libraries, represents enormous accumulated engineering investment. VDR requires new operator libraries, new model conversion tools, new debugging infrastructure, new deployment pipelines. This is multi-year engineering work. The economics justify it; the work is still substantial.

---

10. The Industry Position

10.1 Where We Are

The commercial LLM industry is approximately four years old. Datacenters costing hundreds of billions of dollars are under construction. The hardware going into those facilities is integer-capable, every GPU being installed has INT8 tensor cores delivering 2 $\times$ FP16 throughput.

The software running on that hardware:

- Discards information at every arithmetic operation
- Produces different output from the same input on consecutive runs

- Cannot secure data access structurally
- Spends 80–95% of generated tokens on infrastructure
- Drifts over long operation chains with no remedy
- Cannot perform exact arithmetic
- Requires extensive defensive code against its own special values

These properties are not bugs. They are consequences of the floating-point representation. They cannot be fixed by scaling, optimization, or architectural innovation within the float paradigm.

10.2 Where We Are Heading

The industry trajectory points toward longer outputs (video, extended reasoning), more structured tasks (enterprise integration, automated workflows), higher reliability requirements (regulatory compliance, financial accuracy), and tighter cost constraints (competitive pricing pressure).

Every one of these directions amplifies the problems that floating-point creates and amplifies the advantages that exact integer arithmetic provides. Longer outputs mean longer chains mean more drift. More structured tasks mean more infrastructure tokens to eliminate. Higher reliability means less tolerance for non-determinism. Tighter cost constraints mean the 20–70 $\times$ cost reduction matters more, not less.

The industry is accelerating toward the float wall. The workloads getting investment are the workloads most affected by float's limitations.

10.3 What This Paper Presents

A mechanical analysis. Not advocacy, not speculation, not a sales case. Instruction counts on published hardware. Measured token distributions across task categories. Compound economics that follow from multiplication.

VDR exact integer arithmetic, running on existing GPU hardware, delivers:

- 1.5–2 $\times$ per-token throughput improvement
- 70–98.6% token reduction on structured tasks
- Linear cost scaling versus quadratic
- Exact arithmetic at every step
- Deterministic bit-identical reproduction
- Structural safety requiring zero LLM tokens
- Zero drift at any operation chain length
- A new application development paradigm based on session cloning

These properties compound to 20–70 × total cost reduction on structured workloads that represent the majority of commercial LLM usage.

The hardware exists. The arithmetic is validated. The architecture is specified. The adoption path is incremental. The risk of investigating is bounded. The risk of not investigating is not.

Appendix A: Terminology

Term	Definition
VDR	Value, Denominator, Remainder, an ordered triple of integers representing an exact rational number
D	Denominator, a fixed power of two shared across all values in a pipeline stage
V	Value, the integer numerator, what fits in the current denominator frame
R	Remainder, the exact portion that V could not absorb, stored rather than discarded
Closed	A VDR triple with R = 0, representing the exact rational V/D
Active	A VDR triple with R ≠ 0, carrying exact structure beyond the V slot
divmod	Integer division and modulo, the operation that splits a result into V and R
Basis	The choice of D for a given pipeline stage
Q335	D = 2^335, the high-precision basis used for physics and transcendental computation
Q8, Q16, Q64	D = 2^8, 2^16, 2^64, the ML workload bases for weights, activations, gradients
KB	Knowledge base, scoped storage for facts, rules, constraints, and data at integer addresses
Builtin	A typed primitive operation that executes deterministically, invoked by the LLM via command token
Grammar	A structural template providing fixed output tokens with typed slots for content
SFU	Special Function Unit, the GPU execution unit for transcendentals, 1/16 tensor core throughput

Term	Definition
Barrett reduction	Integer division via precomputed multiplicative inverse, multiply and shift, no division instruction
ULP	Unit in the Last Place, the magnitude of the least significant bit of a float mantissa
IOSE	Input, Output, Side Effects, Properties, the declaration model for every VDR component

Appendix B: Cross-References to VDR Paper Series

Topic	Papers	What they establish
Exact arithmetic foundation	VDR-1, VDR-2, VDR-3, VDR-13	884 tests, 37 domains, zero computation errors
Exact transformer pipeline	VDR-4	softmax, autodiff, training in exact fractions, 198 tests
Knowledge bases and Prolog	VDR-5	scoped KBs, logic engine, provenance
Command tokens and builtins	VDR-6, VDR-10	448 builtins, IOSE model, 25 categories
Data primitives and sessions	VDR-8	7 data primitive types, snapshot/clone/kill lifecycle
Orchestrated inference	VDR-9	4 inference modes, investigation loop, backtracking
Implementation blueprint	VDR-11	65 modules, 5 stages, ~20,500 lines
Grammar compaction	VDR-12	83% compression, 178/179 tests, grammar-directed output
Consolidated specification	VDR-14	complete system in one document
Token economics	VDR-15	85–97% token reduction, crossover analysis
Structural safety	VDR-16	jailbreak impossibility for data access
Alignment	VDR-17	helpful/harmless/honest through structure
GPU performance	VDR-18	Q335 on GPU, 5 concurrent streams
Self-extension	VDR-19	usage as training, rule accumulation

Topic	Papers	What they establish
Deployment	VDR-20	4 runner types, convergence curves
FPGA accelerator	VDR-21	10-core Q335 processor, pre-synthesis
Dedicated silicon	VDR-22	integer-native GPU ASIC design
Functional remainder hardware	VDR-23	FRU unit, adaptive precision in silicon
LLM software	VDR-24	sessions as applications, snapshot as binary
LLM server software	VDR-25	protocol grammars, clone-per-request
Diffusion	VDR-26	zero-drift denoising, exact schedules
Cross-domain applications	VDR-27, VDR-28	20 domains where truncation compounds
Hardware performance	VDR-29	INT8 tensor cores, SFU bypass, per-operation instruction counts
Economics (this paper)	**VDR-30**	**compound cost analysis, adoption path, industry position**

[**HOWL-VDR-30-2026**] Economics of Scale: Floating Point vs Exact Integer ML Models. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-30-2026

[**HOWL-VDR-1-2026**] VDR Arithmetic: Value, Decimal, Remainder. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026

[**HOWL-VDR-2-2026**] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026

[**HOWL-MATH-3-2026**] The Transcendental Hierarchy. Github: https://github.com/ghowland/papers/tree/main/papers/M
MATH-3-2026

[**HOWL-MATH-4-2026**] The Universal Power-of-Two Basis. Github: https://github.com/ghowland/papers/tree/main/pape
MATH-4-2026

[**HOWL-VDR-14-2026**] VDR-LLM-Prolog. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL
VDR-14-2026

[**HOWL-VDR-21-2026**] VDR-LLM-Prolog on FPGA. Github: https://github.com/ghowland/papers/tree/main/papers/VD
VDR-21-2026

[**HOWL-VDR-22-2026**] VDR-LLM-Prolog on Dedicated Silicon. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-22-2026

[**HOWL-VDR-23-2026**] VDR-LLM-Prolog: Functional Remainder Hardware. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-23-2026

[**HOWL-VDR-24-2026**] LLM Software. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-24-2026

[**HOWL-VDR-25-2026**] LLM Server Software. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOV
VDR-25-2026

[**HOWL-VDR-26-2026**] VDR and Diffusion. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL
VDR-26-2026

[**HOWL-VDR-27-2026**] VDR Beyond Language Models. Github: https://github.com/ghowland/papers/tree/main/papers/V
VDR-27-2026

[**HOWL-VDR-28-2026**] Exact Rational Arithmetic for Sequential Computation. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-28-2026

[**HOWL-VDR-29-2026**] VDR in Zig SIMD and GPU Performance versus Floating Point. Github: https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-29-2026